시스템 설계 스터디

실시간 게임 순위표

목차

01 문제 이해 및 설계범위 확정

02 개략적 설계안

03 상세 설계

04 마무리

문제 이해 및 설계범위 확정

기능 요구 사항

- 상위 10명 플레이어 표시
- 특정 사용자의 순위 표시
- 특정 사용자 +-4순위 사용자 표시

* 경기 승리시 +1포인트 / 동점자는 같은 순위

비기능 요구 사항

- 점수는 실시간으로 업데이트
- 일반적인 확장성 가용성 안정성

문제 이해 및 설계범위 확정

규모 추정

- 일간 사용자 == 500만
AVG = 50
MAX= 250 (PEAK=*5 가정)

- 점수 획득 QPS
AVG = 500 (하루 평균 10개 게임 플레이 가정)
MAX = 2500 (PEAK=*5 가정)

- 순위표 가져오기 QPS
약 50 (하루 1회 표시 가정)

개략적 설계

개략적 설계

API

Post /v1/scores - 사용자 순위 갱신
Get /v1/Scores - 상위 10 플레이어 Get
Get /v1/Scores/{:user_id} - 특정 사용자의 순위 Get

개략적 설계안

1. 게임 승리시 클라 → 게임서비스에게 요청
2. 유효성 체크 후 순위표 서비스에게 갱신 요청
3. 순위표 서비스는 해당 사용자 점수 갱신
4. 순위표를 가져올때는 클라이언트가
   순위표 서비스에게 직접 요청

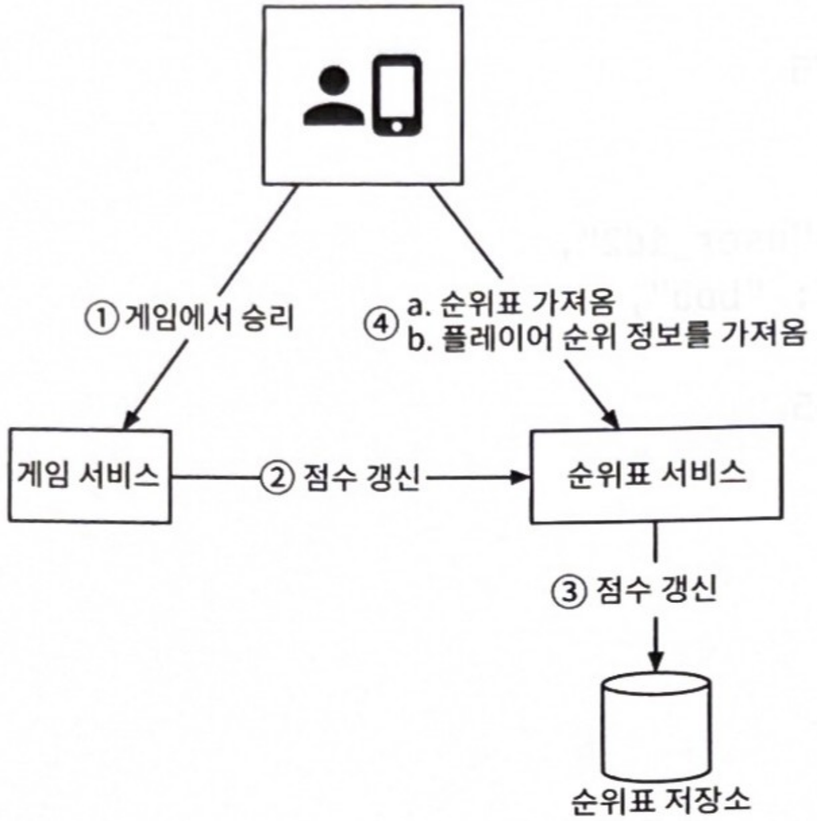


그림 10.2 개략적 설계

개략적 설계

기각안 1

클라↔순위표 서비스 간 직접 갱신

문제점: 중간자 공격 가능성(위변조 가능성)

*클라이언트의 갱신 요청 자체가 없을 수도 있음
→ 게임 서버가 모든 로직을 처리하는 경우

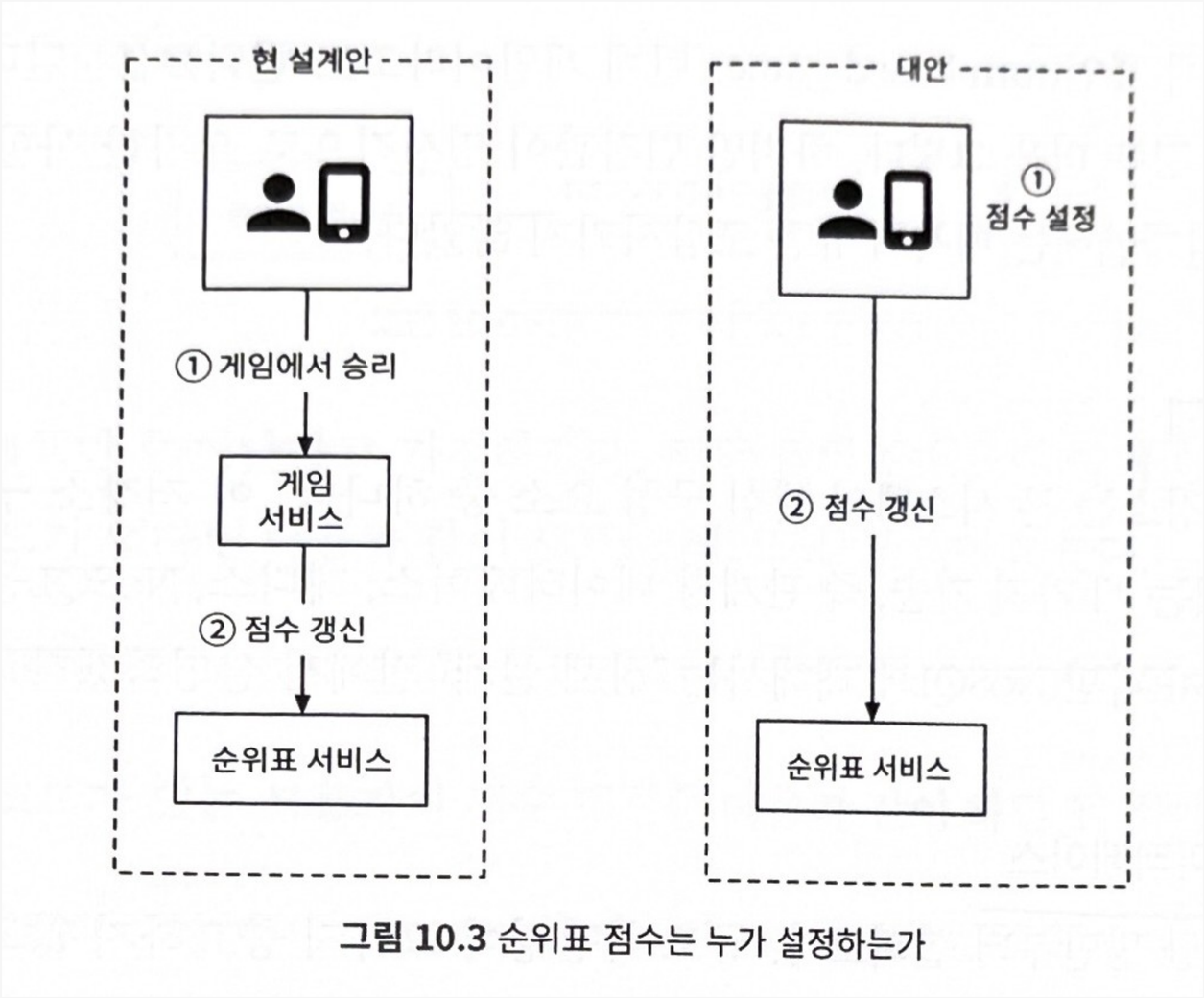


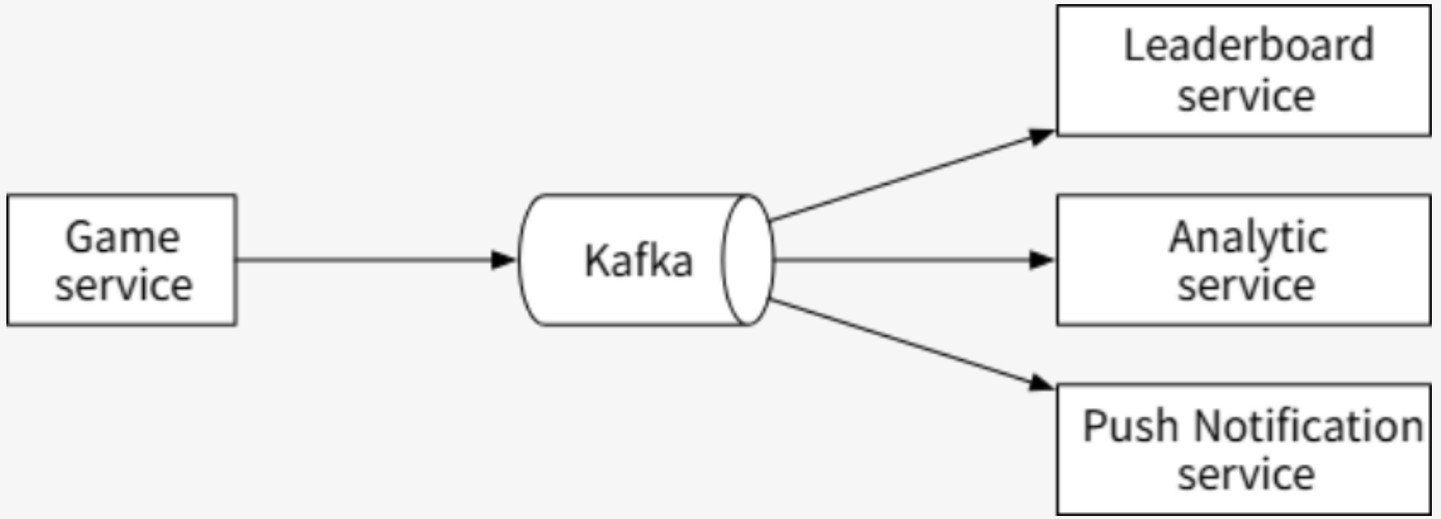
그림 10.3 순위표 점수는 누가 설정하는가

기각안 2 - 메세지 큐

게임 서비스 ↔ 순위표 서버 간 큐 필요성

해당 데이터가 여러 곳에 쓰이고, 턴제 게임이거나 멀티플레이어 게임일 경우 필요할 수도 있음.

그러나, 요청사항에 없으므로 배제.



데이터 모델

선택지
- 관계형 DB
- 레디스
- NoSQL ⇒ 상세 설계에서 대안으로서 다룸

개략적 설계

leaderboard	
user_id	varchar
score	int

관계형 DB

규모 확장성이 적고, 그 인원 수가 적을 때만 가능 == 현재 요구 사항에 맞지 않음

새 사용자 ⇒ insert, 기존 사용자 ⇒ update

문제는 매번 정렬해줘야 한다는 것!
실시간성 유지가 어렵고, 캐싱을 하기도 힘듦

레디스

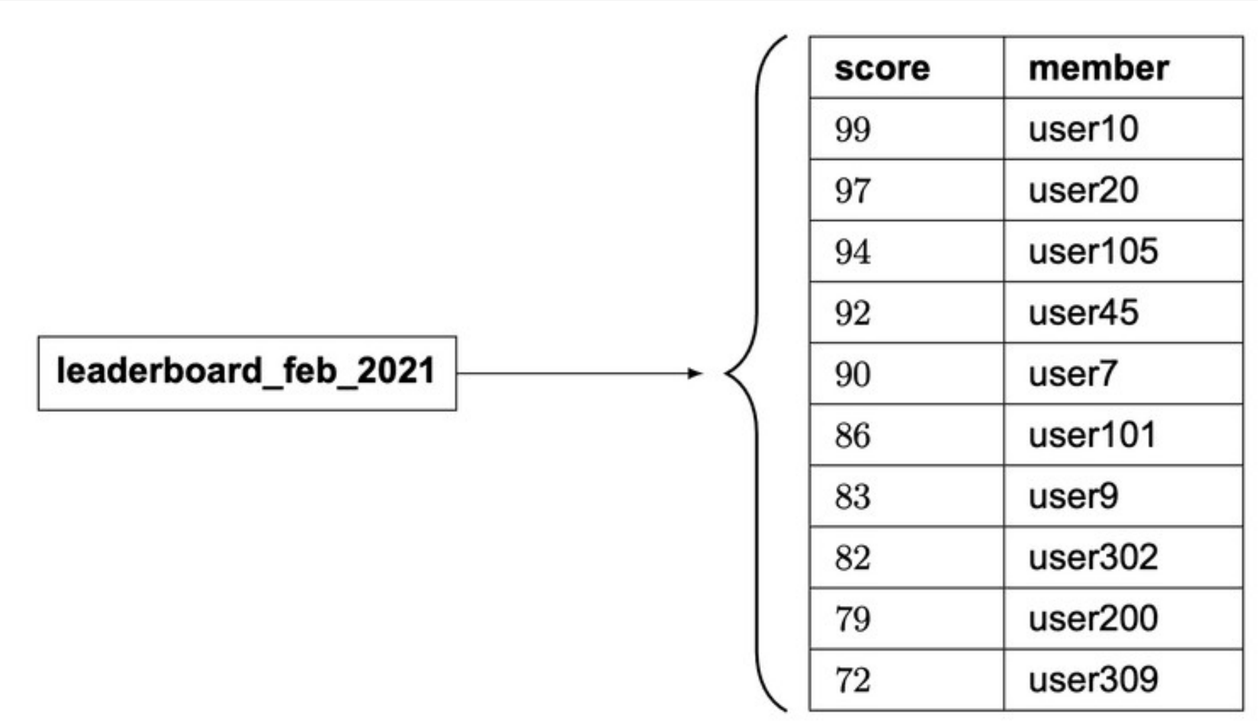
메모리에 올라가는 key-value storage

1. disk level 이 아닌 memory 레벨이라 빠른 읽기/쓰기
2. 정렬집합 제공

정렬 집합

집합과 유사한 자료형
해시 테이블과 스킵 리스트라는 두 가지 자료 구조로 구현

해시 테이블 - 점수 저장
스킵 리스트 - 특정 점수를 딴 사용자들의 목록 저장



개략적 설계

정렬 집합 - 스킵 리스트

여유 공간 없는 MultiLevel B+ Tree indexing 갬성

원본은 내부가 단방향 리스트라 삽입/삭제/갱신이 모두 O(n)
→ 스킵리스트를 이용한 이진 탐색으로 O(log n)으로 감소

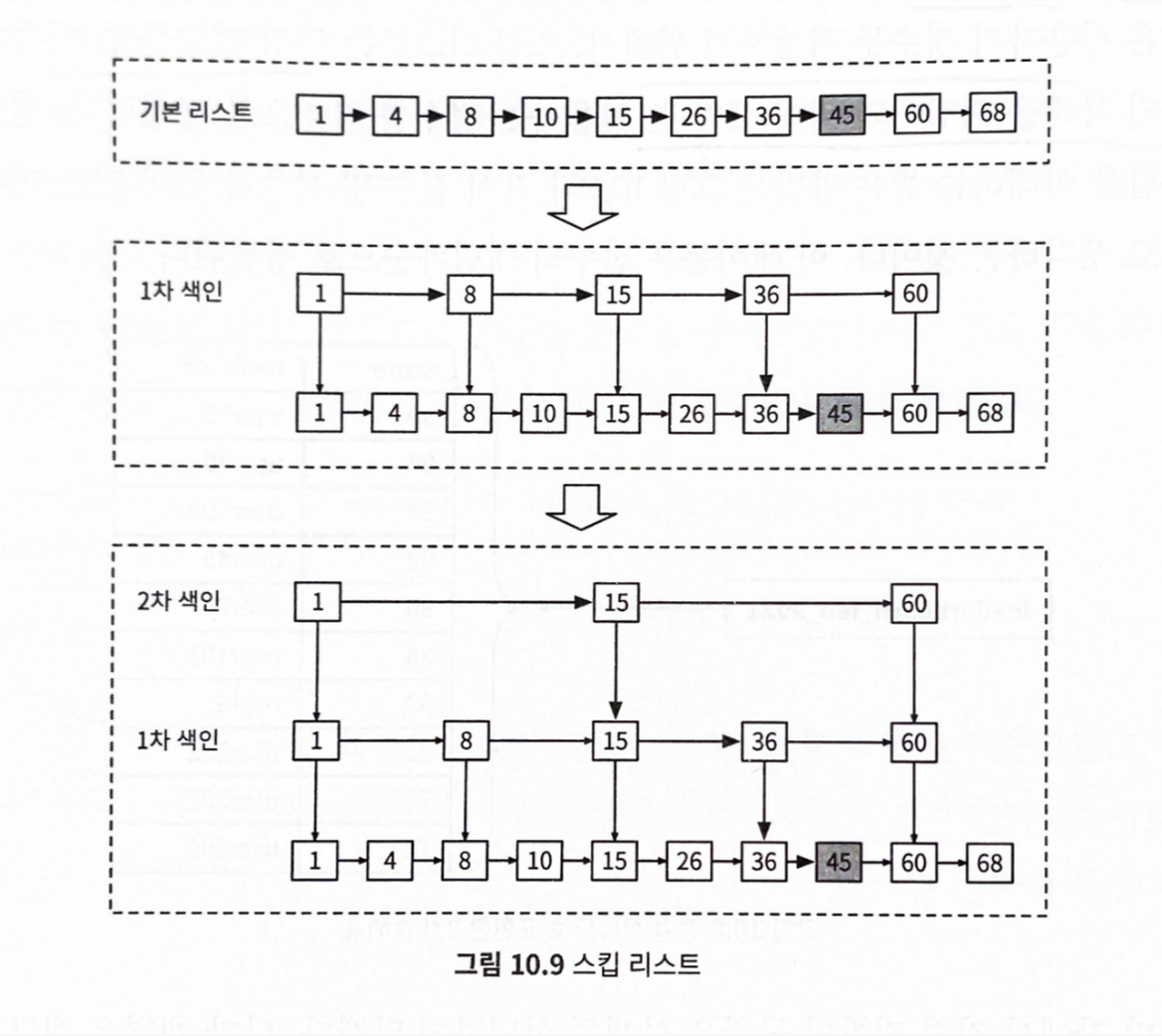


그림 10.9 스킵 리스트

개략적 설계

정렬 집합 - 스킵 리스트

ZADD = O(logn)
ZINCRBY = O(logn)
ZRANGE/ZREVRANGE = O(logn+m)
ZRANK/ZREVRANK = O(logn)

대부분 탐색시간!

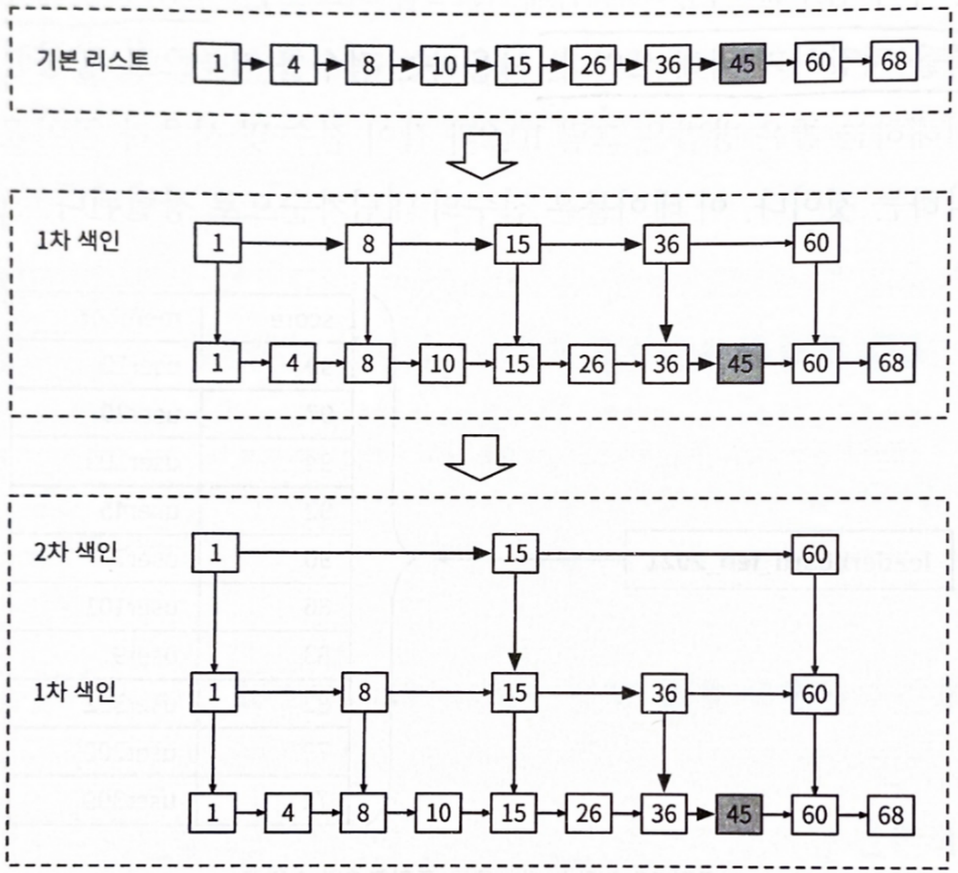


그림 10.9 스킵 리스트

저장소 요구사항

최소 ID/점수 저장
최악의 시나리오 ⇒ 2500만명 전부 1회 이상 플레이
(ID 24B + Score 2B)*2500만 = 650MB ⇒ 무리 없이 저장 가능!

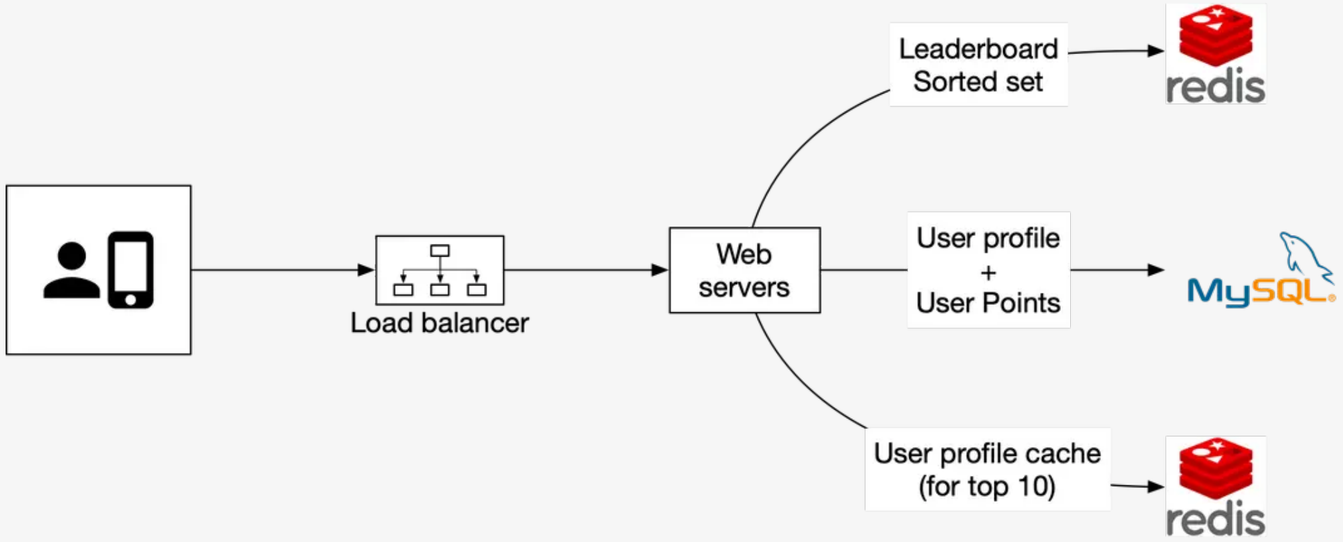
QPS 2500/초 ⇒ 감당 가능!

BUT 영속성 Issue 있음
따라서 디스크 영속적으로 저장 옵션 사용
or
읽기 사본 레플리카 생성
or
관계 DB로 원 data를 저장해서 복구해서 쓸 수 있게 함

상세 설계

클라우드 사용 여부 - 자체 서비스 사용

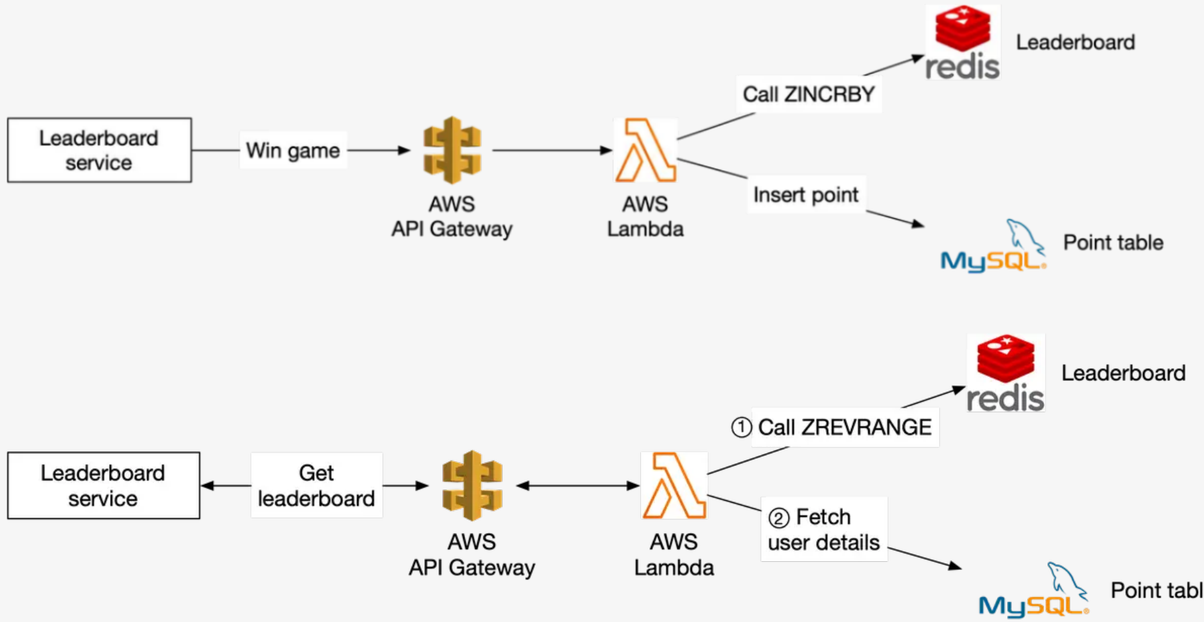
기본적으로 직전에 다룬 것과 비슷
- 리더보드는 레디스로 저장
- 상세정보는 MySQL에 따로
- top10은 자주 쓰이니 캐싱



클라우드 사용 여부 - AWS 사용

아마존 GateWay + AWS 람다 조합

GateWay를 이용해 적절한 백엔드에 연결하고, 서버리스인 람다를 이용해
트래픽에 따라 자동 확장



레디스 규모 확장

500만 DAU→ 5억 DAU (로블록스가 1억 5천이 안되는데 나올리가...읍읍)

데이터 샤딩 방안
-고정 파티션
-해시 파티션

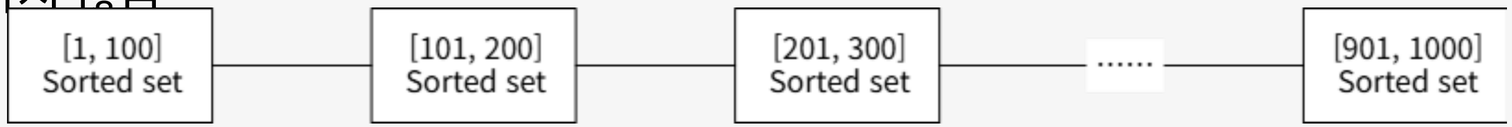
고정 파티션

순위표 내 점수 범위에 따라 파티션을 나누기

분포를 알아낼 수 있다면, 이를 이용해 최대한 고르게 나누는 게 좋음. *

입력/갱신시 어떤 샤드에 접근해야하는지는 id-score 2차 캐시가 있으면 좋음**

*단 이 경우 애플리케이션이 샤딩해야함
**일반적인 캐싱을 말하는 것 같지만 확실하지 않음

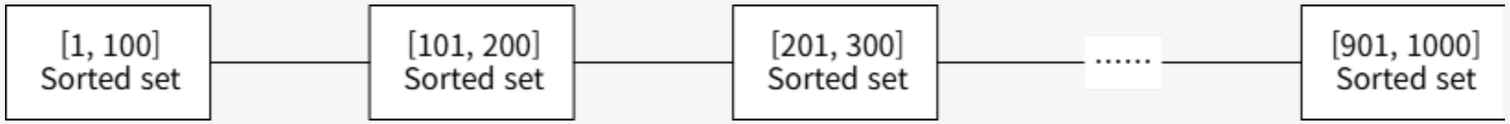


예시는 fixed 사이즈지만 그럴 필요는 없음

고정 파티션 2

샤드 임계값에서 삭제-생성 필요함 유의

상위 10명을 구하는 방법이나
특정 사용자의 순위를 구하는 방법 역시 큰 무리 없이 진행 가능

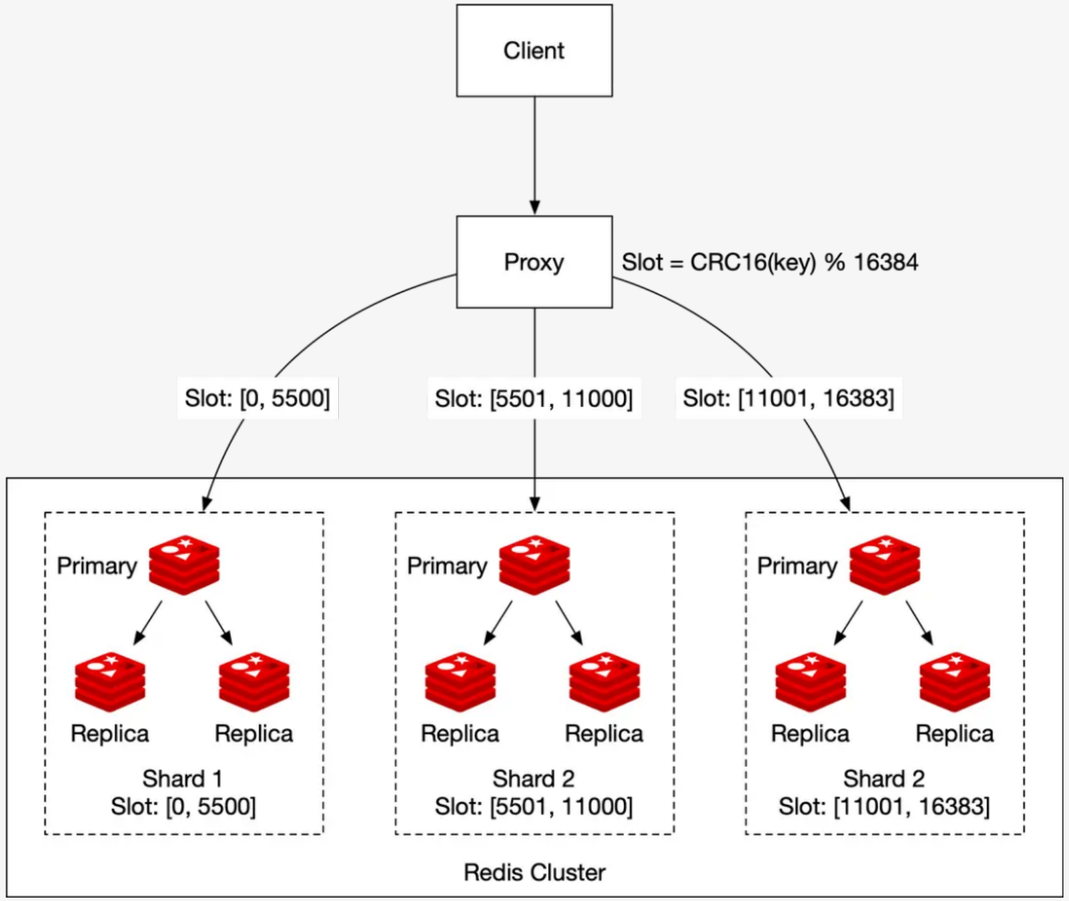


예시는 fixed 사이즈지만 그럴 필요는 없음

해시 파티션

레디스 클러스터에 제공되는 자동 샤딩 방법 사용
CRC16(key)%16384로 구함

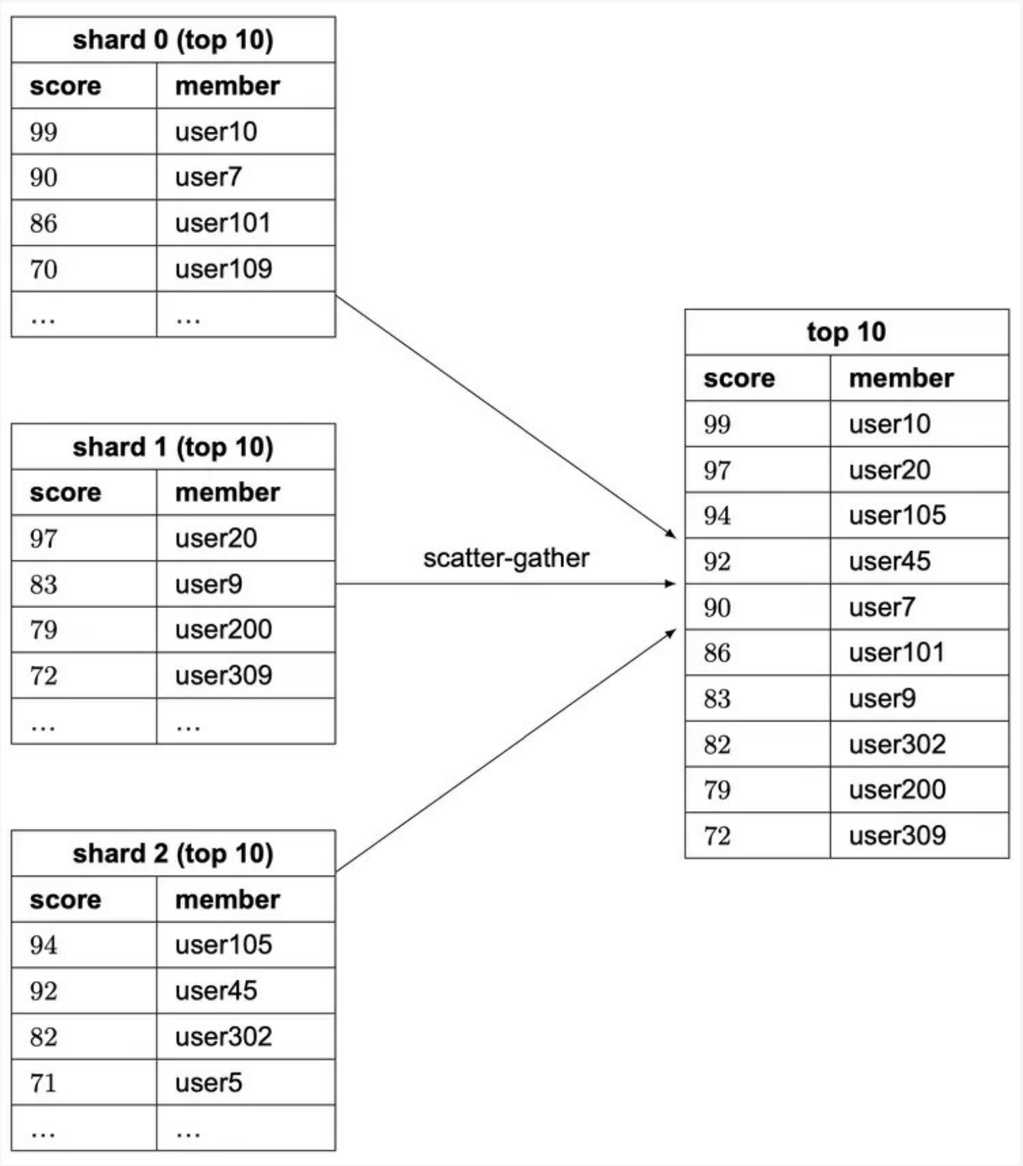
점수 갱신은 매우 쉽게 가능



상세 설계

해시 파티션 BUT

1. 상위 n명을 구하기 매우 어려움
각 샤드에서 n명씩 모두 구하고 그걸 다시 재정렬해서 구해야함
당연하지만 모든 샤드의 데이터가 다 들어와야만 구할 수 있음
2. 특정 사용자 순위를 구할 쉬운 방법이 없음



상세 설계

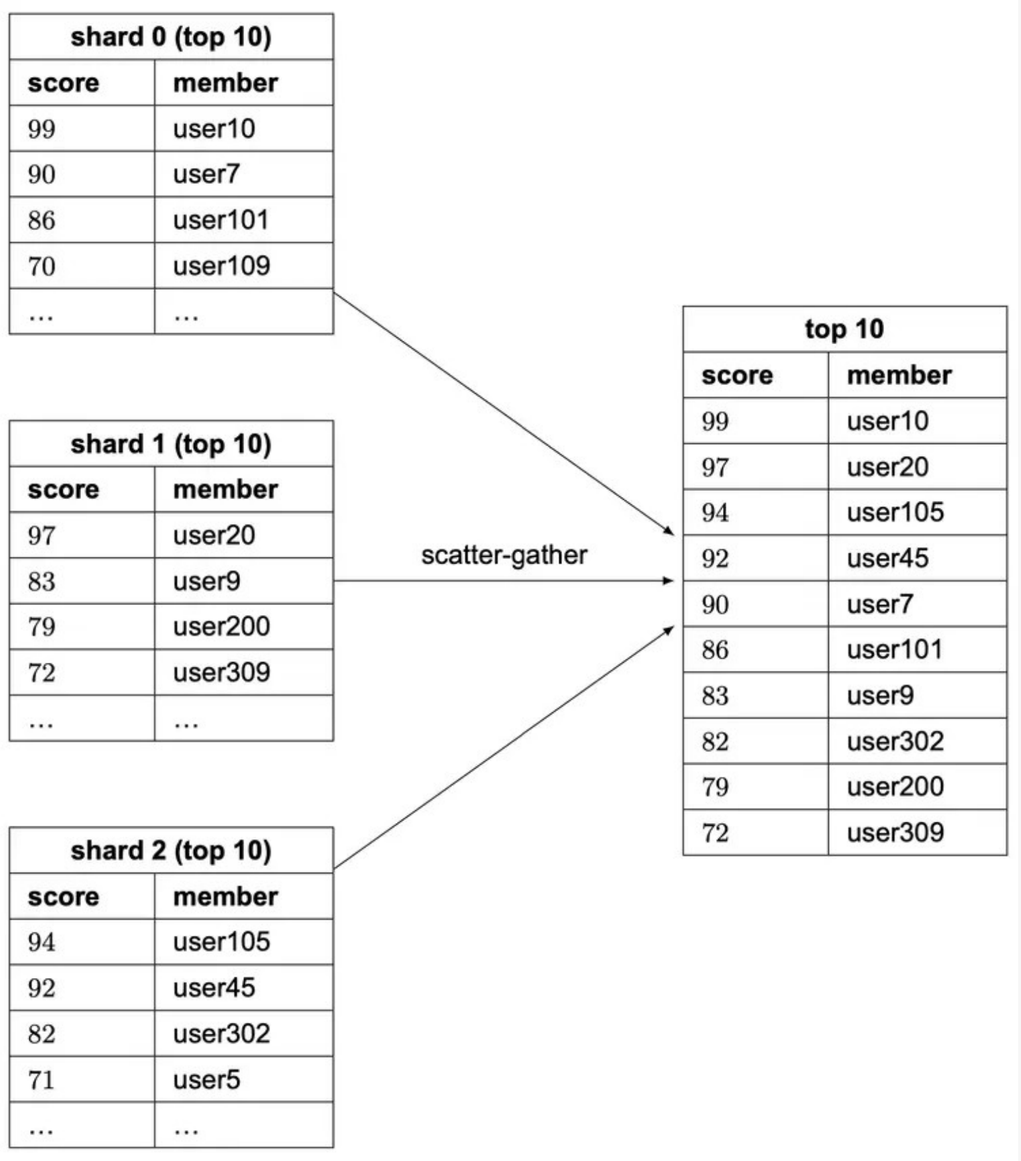
레디스 노드 크기 조정

레디스 노드의 크기를 조정할 때는 여러가지를 고려해야함

쓰기가 많을 경우 ⇒ 2배 많은 메모리 (스냅샷 저장용)

이를 위해 redis-benchmark라는 도구로
多 클라 多질의 상황에서 하드웨어가 초당 얼마나 처리할 수 있는지
측정 가능

(책에서 자세히 다루진 않음)



대안: No SQL

요구사항은 크게 2가지
1. 쓰기 연산 최적화
2. 정렬 용이

DynamoDB/카산드라/MongoDB 등이 후보

책에서는 이중 DynamoDB를 서술

DynamoDB

AWS에서 제공하는 완전 관리형(Fully Managed) 분산 NoSQL 데이터베이스

기본키 이외 질의를 효율적으로 할 수 있도록 전역 보조 색인 제공
이때 전역 보조인덱스 (GSI)는 부모 속성들로 구성되지만 기본키는 부모테이블과 다름.

GSI→ 다른 Primary key를 가진 관계 DB Materialized View 생성과 유사



DynamoDB

원본은 user_id를 Primary Key로 가짐

이 경우 정렬도 안되어 있으므로 모든 테이블을 뒤져야 상위 10명을 알아낼 수 있음.

Primary key	Attributes			
user_id	score	email	profile_pic	leaderboard_name
lovelove	309	love@test.com	https://cdn.example/3.png	chess#2020-02
i_love_tofu	209	test@test.com	https://cdn.example/p.png	chess#2020-02
golden_gate	103	gold@test.com	https://cdn.example/2.png	chess#2020-03
pizza_or_bread	203	piz@test.com	https://cdn.example/31.png	chess#2021-05
ocean	10	oce@test.com	https://cdn.example/32.png	chess#2020-02
…	…	…	…	…

상세 설계

DynamoDB

GSI ⇒leader board_name은 Partiotion Key로, 그리고 Sort key로 Score를 선정하여 Primary key*를 구성

왜 이런식으로 표기했는지는 의문→

*관계 DB의 PK랑 중복 가능하다는 점에서 다름

Global Secondary Index		Attributes		
Partition key (PK)	**Sort key (score)**	**user_id**	**email**	**profile_pic**
chess#2020-02	309	lovelove	love@test.com	https://cdn.example/3.png
chess#2020-02	209	i_love_tofu	test@test.com	https://cdn.example/p.png
chess#2020-03	103	golden_gate	gold@test.com	https://cdn.example/2.png
chess#2020-02	203	pizza_or_bread	piz@test.com	https://cdn.example/31.png
chess#2020-02	10	ocean	oce@test.com	https://cdn.example/32.png
…	…	…	…	…

DynamoDB

DynamoDB는 안정해시를 이용해 PK로 여러 노드에 분산함
따라서
Issue 1: 최근 한달치 데이터가 모두 1개의 노드에 저장
Issue 2: 따라서 해당 데이터가 핫 파티션이 되어 버림

Global Secondary Index		Attributes		
Partition key (PK)	**Sort key (score)**	**user_id**	**email**	**profile_pic**
chess#2020-02	309	lovelove	love@test.com	https://cdn.example/3.png
chess#2020-02	209	i_love_tofu	test@test.com	https://cdn.example/p.png
chess#2020-03	103	golden_gate	gold@test.com	https://cdn.example/2.png
chess#2020-02	203	pizza_or_bread	piz@test.com	https://cdn.example/31.png
chess#2020-02	10	ocean	oce@test.com	https://cdn.example/32.png
…	…	…	…	…

DynamoDB

해결법은... user_id%n으로 파티션 번호를 또 추가하기
당연하지만 읽기 쓰기가 복잡해짐...

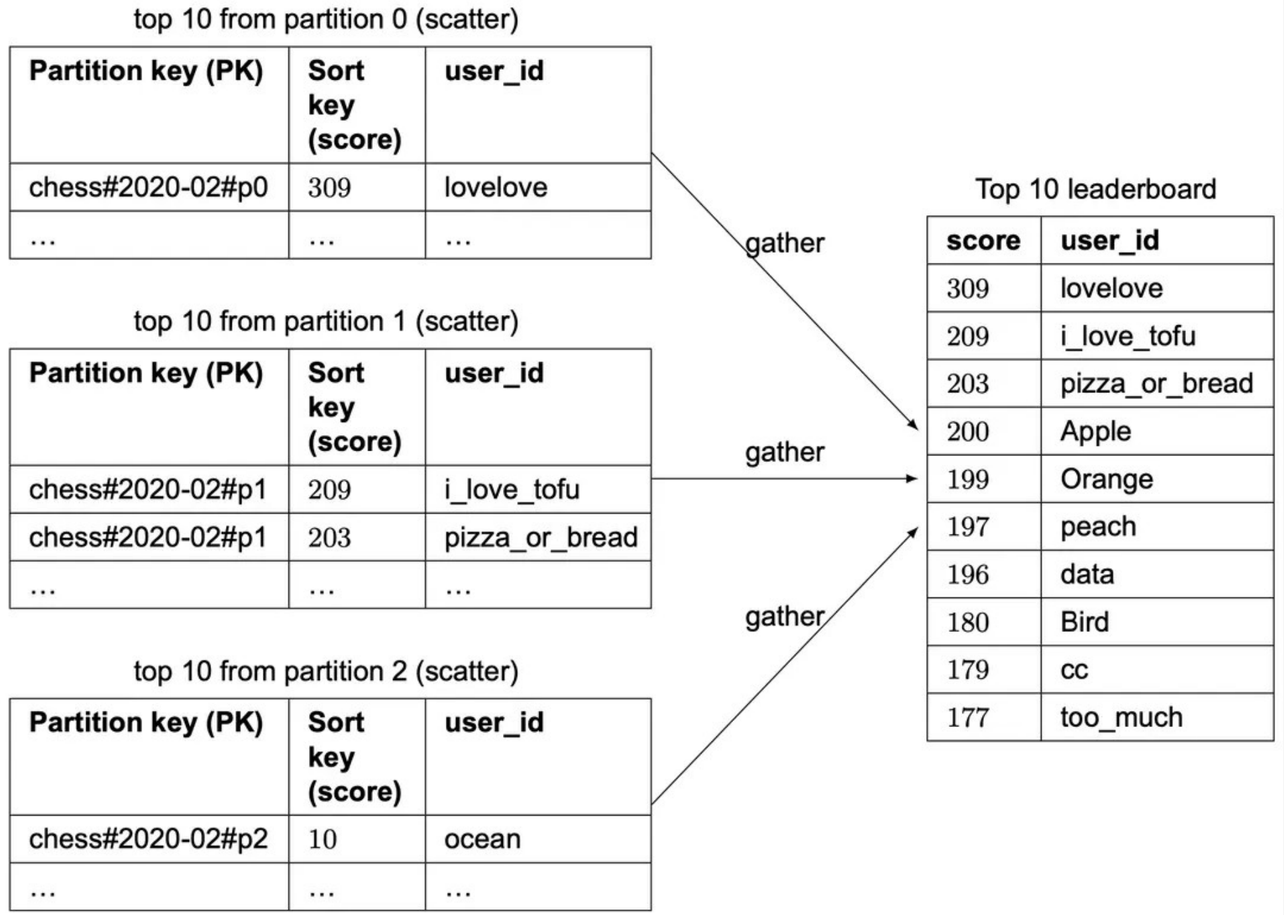
Global Secondary Index		Attributes		
Partition key (PK)	**Sort key (score)**	**user_id**	**email**	**profile_pic**
chess#2020-02#p0	309	lovelove	love@test.com	https://cdn.example/3.png
chess#2020-02#p1	209	i_love_tofu	test@test.com	https://cdn.example/p.png
chess#2020-03#p2	103	golden_gate	gold@test.com	https://cdn.example/2.png
chess#2020-02#p1	203	pizza_or_bread	piz@test.com	https://cdn.example/31.png
chess#2020-02#p2	10	ocean	oce@test.com	https://cdn.example/32.png
…	…	…	…	…

상세 설계

DynamoDB

여전한 문제는
상위 10명 구하기가 힘들다!
특정 순위 구하기가 힘들다!

BUT 데이터가 충분히 크면 백분위 수 정도는 유사하게 구할 수 있음

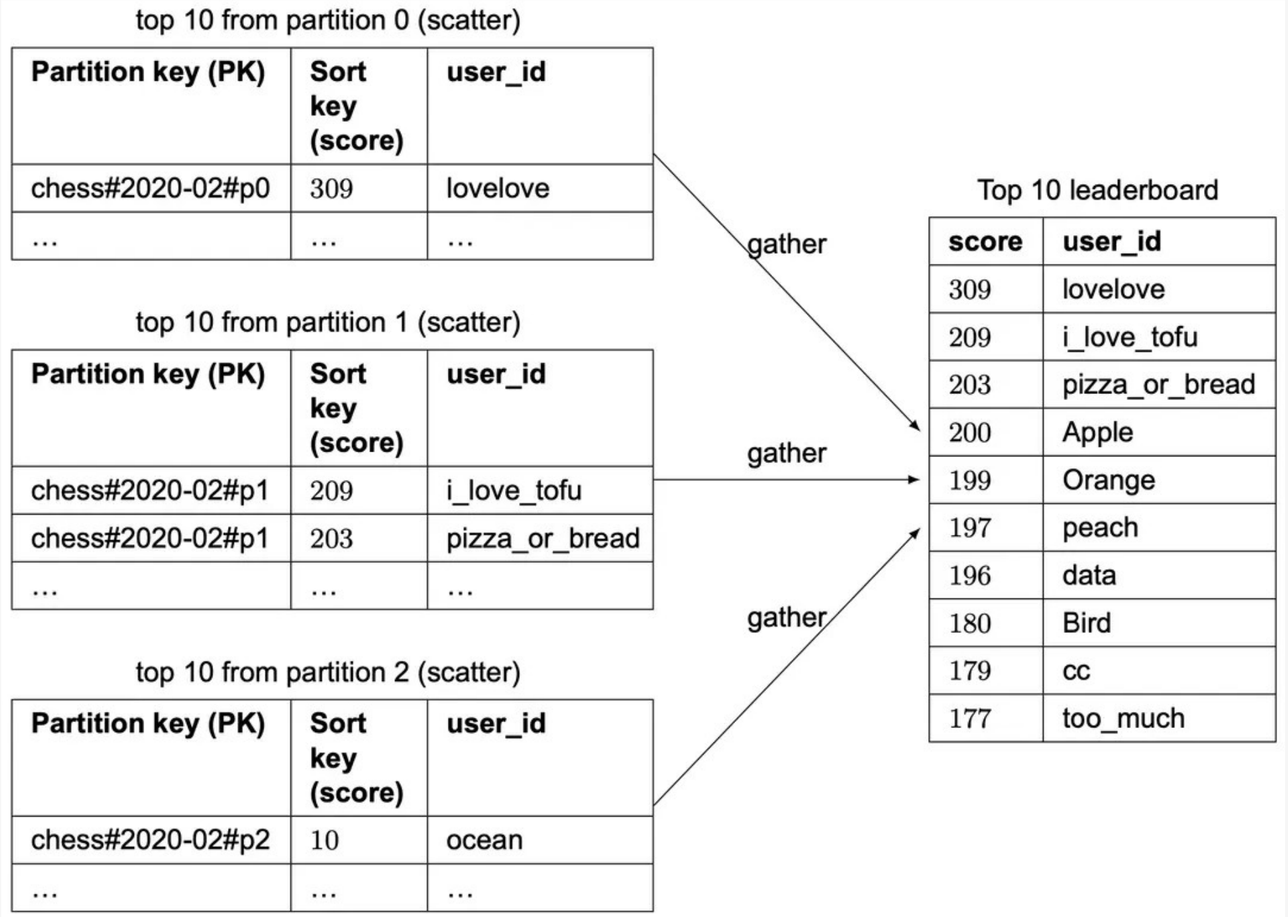


상세 설계

DynamoDB

10th percentile = score < 100
20th percentile = score < 500
...
90th percentile = score < 6500

이런 크론 작업을 이용해 속도를 높이는 꼼수도 소개함



마무리

마무리

더 빠른 조회 및 동점자 순위 판정

레디스 해시를 이용!

1. 사용자 정보 등을 해시에 저장해서 빠르게 가져올 수 있음 (ex 프로필 이미지 등)
2. 해시에 타임스탬프를 저장해서 동점자의 우선순위를 정할 수도 있음

단, 해시는 순서가 없으므로 sorted set은 여전히 필요함

마무리

시스템 장애 복구

초반에 언급한대로, 사용자가 게임에 이길때마다 상세 정보를 MySQL에 저장
⇒이를 이용해 복구

감사합니다